Flight Price Tracker Bot — Full Architecture & Implementation Plan

Project Overview

A fully automated flight price tracking system built on n8n that includes a Telegram/WhatsApp conversational bot, hourly and daily price checks via the Amadeus Flight Offers API, Google Sheets data storage, and multi-channel alerts (Telegram, WhatsApp, Gmail). The system handles multi-user tracking, nearby airport detection, and smart drop alerts after 3+ days of monitoring.

System Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          USER CHANNELS                                   │
│   [Telegram Bot]   [WhatsApp via Twilio]   [Gmail Alerts]                │
└──────────┬──────────────────┬──────────────────┬──────────────────────────┘
           │                  │                  │
           ▼                  ▼                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                         n8n AUTOMATION ENGINE                            │
│                                                                          │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐       │
│  │ Workflow 1:      │  │ Workflow 2:      │  │ Workflow 3:      │       │
│  │ Conversation     │  │ Hourly Price     │  │ Daily Summary    │       │
│  │ Bot              │  │ Monitor          │  │ Digest           │       │
│  └──────────────────┘  └──────────────────┘  └──────────────────┘       │
└──────────┬──────────────────┬──────────────────┬──────────────────────────┘
           │                  │                  │
           ▼                  ▼                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                        EXTERNAL SERVICES                                 │
│   [Amadeus Flight Offers API]   [Google Sheets]   [Twilio API]           │
└──────────────────────────────────────────────────────────────────────────┘
```

Directory & File Structure

```
flight-price-tracker/
├── n8n-workflows/
│   ├── workflow-1-conversation-bot.json       ← Import into n8n
│   ├── workflow-2-hourly-price-monitor.json    ← Import into n8n
│   └── workflow-3-daily-digest.json           ← Import into n8n
├── google-sheets/
│   └── flight-tracker-schema.md              ← Sheet column definitions
├── config/
```

```
│   └── env-variables.md                      ← All API keys & tokens
├── prompts/
│   └── bot-conversation-messages.md           ← All bot message templates
└── README.md
```

Workflow 1 — Conversational Bot (Telegram + WhatsApp)

Purpose

Handles the full onboarding conversation with the user. Collects route details, travel dates, alert preferences, and triggers the first price search.

Trigger

`Telegram Trigger` node (webhook-based) — fires on every incoming message.

Conversation State Machine

The bot uses Google Sheets (a "Sessions" tab) to store each user's current step. Each Telegram `chat_id` maps to a row with a `step` column.

```
step_0  → Send welcome + ask: "Do you want email alerts? Reply YES or NO"
step_1  → (If YES) Ask: "Enter your Gmail address"
          (If NO)  Skip to step_2
step_2  → Ask: "From which city/airport? (e.g. Toronto or YYZ)"
step_3  → Ask: "To which city/airport? (e.g. Mumbai or BOM)"
step_4  → Ask: "Travel date? (format: YYYY-MM-DD)"
step_5  → Ask: "Return date? (or type ONEWAY for one-way)"
step_6  → Ask: "How many passengers? (adults)"
step_7  → Ask: "Check nearby airports too? Reply YES or NO"
step_8  → Ask: "Alert me when price drops by:
                 1) 5%  2) 10%  3) 15%  4) Custom ($amount)"
step_9  → Confirm all details + trigger first price search
step_10 → ACTIVE (tracking in progress, listening for /stop or /status)
```

n8n Node Flow — Workflow 1

```
[Telegram Trigger]
      │
      ▼
[Google Sheets - Read Session]
  (Lookup chat_id in Sessions tab)
      │
      ▼
[Switch Node — current step?]
      │
  ┌───┴──────────────────────────┐
  │                              │
step_0..8                    step_10
  │                              │
```

```
[Set Node - save answer]    [Switch - command?]
[Update Google Sheets]       /stop → [Mark inactive]
      |                     /status → [Fetch & reply]
      ▼
[IF — is step 9?]
  YES → [HTTP Request - Amadeus OAuth token]
        [HTTP Request - Amadeus Flight Offers Search]
        [Format Results Node]
        [Google Sheets - Write to Tracker tab]
        [Telegram - Send price results]
        [IF - email_alerts = YES]
          YES → [Gmail - Send confirmation + current prices]
        [Update step to step_10]
  NO  → [Telegram - Send next question]
```

Amadeus API Call (step 9)

Endpoint: `GET https://test.api.amadeus.com/v2/shopping/flight-offers`

Parameters to build dynamically from collected user data:

```
originLocationCode     = {{from_airport_code}}
destinationLocationCode = {{to_airport_code}}
departureDate          = {{travel_date}}
returnDate             = {{return_date}} (omit if ONEWAY)
adults                 = {{passengers}}
max                    = 5  (return top 5 cheapest options)
currencyCode           = CAD (or user's preferred currency)
```

Nearby Airports Logic:
If user answered YES to nearby airports, build a lookup map:

```
// In a Code node (JavaScript)
const airportGroups = {
  "YYZ": ["YYZ", "YTZ", "YHM", "YKF"],   // Toronto area
  "YVR": ["YVR", "YXX"],                  // Vancouver area
  "JFK": ["JFK", "LGA", "EWR"],           // New York area
  "LHR": ["LHR", "LGW", "STN", "LTN"],    // London area
  "BOM": ["BOM", "PNQ"],                  // Mumbai area
  // Expand as needed
};

const userInput = $json.from_airport.toUpperCase();
const matchedCode = Object.keys(airportGroups).find(k =>
  userInput.includes(k) || userInput.includes("TORONTO")
  // add city-name matching logic
);
return airportGroups[matchedCode] || [userInput];
```

Run separate Amadeus calls per airport code and merge/sort results by price.

Workflow 2 — Hourly Price Monitor

Purpose

Runs every hour. Reads all active tracking rows from Google Sheets, fetches live prices from Amadeus, compares with stored prices, and sends alerts if price drops exceed the user's threshold.

Trigger

`Schedule Trigger` → Every 1 hour (cron: `0 * * * *`)

n8n Node Flow — Workflow 2

```
[Schedule Trigger - every 1 hour]
      |
      ▼
[Google Sheets - Read all rows from Tracker tab]
  Filter: status = "active"
      |
      ▼
[Split In Batches node]
  (Process each tracked route individually)
      |
      ▼
[HTTP Request - Amadeus OAuth token]
      |
      ▼
[HTTP Request - Amadeus Flight Offers Search]
  (Use stored from/to/date from current row)
      |
      ▼
[Code Node — Extract cheapest price]
  const offers = $json.data;
  const cheapest = Math.min(...offers.map(o => parseFloat(o.price.grandTotal)));
  return { current_price: cheapest };
      |
      ▼
[IF — current_price < last_price * (1 - threshold/100)]
  YES →
    [IF — tracking_days >= 3 AND current_price < lowest_ever_price]
      YES → Send "⊠ BEST DEAL ALERT" (special priority message)
      NO  → Send "⊠ Price Drop Alert" (standard drop message)
    [Telegram - Send alert to chat_id]
    [IF — email_alerts = YES]
      YES → [Gmail - Send price drop email]
    [IF — whatsapp_number is set]
      YES → [Twilio WhatsApp - Send alert]
    [Google Sheets - Update last_price, lowest_price, last_checked]
  NO →
    [Google Sheets - Update last_price, last_checked only]
      |
```

```
      ▼
[Continue loop to next row]
```

Price Drop Logic (Code Node)

```
// Inputs from Google Sheets row
const lastPrice = parseFloat($json.last_price);
const currentPrice = parseFloat($json.current_price);
const thresholdPercent = parseFloat($json.alert_threshold_percent);
const thresholdFixed = parseFloat($json.alert_threshold_fixed || 0);
const lowestPrice = parseFloat($json.lowest_price);
const trackingSince = new Date($json.tracking_since);
const now = new Date();
const daysDiff = Math.floor((now - trackingSince) / (1000 * 60 * 60 * 24));

const percentDrop = ((lastPrice - currentPrice) / lastPrice) * 100;
const isSignificantDrop = percentDrop >= thresholdPercent || (currentPrice <= lastPr
const isNewLowest = currentPrice < lowestPrice;
const isMatureDeal = daysDiff >= 3 && isNewLowest;

return {
  currentPrice,
  percentDrop: percentDrop.toFixed(2),
  isSignificantDrop,
  isNewLowest,
  isMatureDeal,
  daysDiff
};
```

Workflow 3 — Daily Summary Digest

Purpose

Runs once per day (9 AM user time). Sends a formatted price summary for all active routes belonging to each user.

Trigger

`Schedule Trigger` → Daily at 9:00 AM (cron: `0 9 * * *`)

n8n Node Flow — Workflow 3

```
[Schedule Trigger - 9 AM daily]
      |
      ▼
[Google Sheets - Read Tracker tab]
  Filter: status = "active"
      |
      ▼
[Code Node — Group rows by chat_id]
```

```
      |
      ▼
[Loop over each unique user]
      |
      ▼
[HTTP Request - Amadeus (for each route of this user)]
      |
      ▼
[Code Node — Format digest message]
  Build multi-route summary:
  "⊠ Daily Flight Price Report - April 18
   ────────────────────────
   ✈ Toronto (YYZ) → Mumbai (BOM)
   Apr 18: CAD 812  |  Yesterday: CAD 845
   Lowest Ever: CAD 795  |  Tracking: 5 days
   ────────────────────────
   ✈ Toronto (YYZ) → Delhi (DEL)
   Apr 18: CAD 940  |  Yesterday: CAD 940
   Lowest Ever: CAD 910  |  Tracking: 2 days"
      |
      ├──► [Telegram - Send digest to chat_id]
      ├──► [IF email = YES] → [Gmail - Send daily email]
      └──► [IF whatsapp = YES] → [Twilio WhatsApp - Send digest]
```

Google Sheets Structure

Sheet 1: `Sessions` (Conversation State)

Column	Type	Description
`chat_id`	String	Telegram or WhatsApp user ID
`platform`	String	"telegram" or "whatsapp"
`step`	Number	Current step in onboarding (0–10)
`temp_data`	JSON String	Partial answers during onboarding
`last_active`	Timestamp	Last message time

Sheet 2: `Tracker` (Active Routes)

Column	Type	Description
`tracker_id`	String	UUID (auto-generated)
`chat_id`	String	User's Telegram/WhatsApp ID
`platform`	String	"telegram" or "whatsapp"
`from_airport`	String	IATA code e.g. YYZ
`to_airport`	String	IATA code e.g. BOM

Column	Type	Description
`travel_date`	Date	YYYY-MM-DD
`return_date`	Date	YYYY-MM-DD or "ONEWAY"
`passengers`	Number	Adult count
`check_nearby`	Boolean	TRUE/FALSE
`initial_price`	Number	Price at time of first search
`last_price`	Number	Most recently fetched price
`lowest_price`	Number	Lowest price seen so far
`last_checked`	Timestamp	Last Amadeus fetch time
`tracking_since`	Date	When tracking started
`alert_threshold_percent`	Number	e.g. 10 (for 10%)
`alert_threshold_fixed`	Number	e.g. 50 (for $50 drop)
`email_alerts`	Boolean	TRUE/FALSE
`email_address`	String	user@gmail.com
`whatsapp_number`	String	+14161234567
`status`	String	"active" or "stopped"
`currency`	String	CAD, USD, INR, etc.

Sheet 3: `PriceHistory` (For Trend Analysis)

Column	Type	Description
`tracker_id`	String	References Tracker sheet
`timestamp`	Timestamp	When price was fetched
`price`	Number	Price at this moment
`airline`	String	Carrier name
`flight_numbers`	String	e.g. "AC872, AI101"

Environment Variables (n8n Credentials)

```
AMADEUS_API_KEY          = your-amadeus-api-key
AMADEUS_API_SECRET       = your-amadeus-api-secret
AMADEUS_BASE_URL         = https://test.api.amadeus.com  (switch to production later

TELEGRAM_BOT_TOKEN       = your-telegram-bot-token
TELEGRAM_WEBHOOK_URL     = https://your-n8n-instance.com/webhook/telegram
```

```
TWILIO_ACCOUNT_SID       = your-twilio-sid
TWILIO_AUTH_TOKEN        = your-twilio-auth-token
TWILIO_WHATSAPP_FROM     = whatsapp:+14155238886  (Twilio sandbox number)

GMAIL_OAUTH_CLIENT_ID    = your-gmail-oauth-id
GMAIL_OAUTH_CLIENT_SECRET = your-gmail-oauth-secret

GOOGLE_SHEETS_ID         = your-google-sheet-id
GOOGLE_SHEETS_CREDENTIALS = your-service-account-json
```

Step-by-Step Implementation Plan

Phase 1 — Setup & Credentials (Day 1)

1.1 Create Telegram Bot

- Open Telegram → search `@BotFather`
- Run `/newbot` → give name and username
- Copy the `API Token`
- Set webhook: `https://api.telegram.org/bot{TOKEN}/setWebhook?url={N8N_WEBHOOK_URL}`

1.2 Amadeus Developer Account

- Go to developers.amadeus.com
- Register → Create App → Copy API Key + Secret
- Use `test.api.amadeus.com` for development
- OAuth endpoint: `POST https://test.api.amadeus.com/v1/security/oauth2/token`
  - Body: `grant_type=client_credentials&client_id=KEY&client_secret=SECRET`

1.3 Google Sheets Setup

- Create a new Google Sheet with 3 tabs: `Sessions`, `Tracker`, `PriceHistory`
- Add all columns per schema above
- Enable Google Sheets API in Google Cloud Console
- Create a Service Account → Download JSON key
- Share the Sheet with the service account email

1.4 Twilio WhatsApp (Optional)

- Sign up at twilio.com
- Enable WhatsApp Sandbox (for testing)
- For production: apply for WhatsApp Business approval
- Note your sandbox number and credentials

1.5 Gmail OAuth in n8n

- In n8n → Credentials → New → Gmail OAuth2
- Follow Google OAuth flow
- Grant access to send emails

Phase 2 — Build Workflow 1: Conversation Bot (Days 2–3)

Step 1: Create `Telegram Trigger` node

- Set webhook URL → test with a message

Step 2: Create Google Sheets "Read Session" node

- Operation: `Read Row`
- Match by `chat_id` column

Step 3: Build Switch node with 11 cases (step_0 through step_10)

Step 4: For each step, create:

- A "Send Question" Telegram node
- A "Save Answer" Google Sheets Update node
- An "Advance Step" logic node

Step 5: At step_9, build the Amadeus API call:

- First call: HTTP Request → Amadeus OAuth → capture `access_token`
- Second call: HTTP Request → Flight Offers Search
- Parse response with Code node
- Format message with prices
- Send Telegram reply
- Write full row to Tracker sheet

Step 6: Add /stop and /status command handlers in step_10

Phase 3 — Build Workflow 2: Hourly Monitor (Day 4)

Step 1: Create Schedule Trigger (every 1 hour)

Step 2: Google Sheets Read node → filter `status = active`

Step 3: Split In Batches node (batch size: 1)

Step 4: Amadeus OAuth + Flight Offers Search per row

Step 5: Code node for price drop calculation

Step 6: IF node branching:

- Significant drop → send alert to Telegram + WhatsApp + Gmail
- 3+ days + new lowest → send priority "Best Deal" alert
- No drop → just update `last_checked`

Step 7: Google Sheets update nodes (last_price, lowest_price, last_checked, PriceHistory append)

Phase 4 — Build Workflow 3: Daily Digest (Day 5)

Step 1: Schedule Trigger (daily 9 AM)

Step 2: Read all active rows

Step 3: Code node to group rows by `chat_id`

Step 4: Loop node + Amadeus call per route

Step 5: Code node to build formatted digest string

Step 6: Send via Telegram, Gmail, WhatsApp per user preference

Phase 5 — Testing (Day 6)

- Test bot conversation flow end-to-end in Telegram
- Manually trigger hourly workflow → verify price fetch and Google Sheets update
- Simulate a price drop (change `last_price` in sheet to higher value) → verify alert fires
- Test `/stop` command → verify status updates to "stopped"
- Test daily digest formatting
- Switch Amadeus to production environment after testing

Phase 6 — Deployment (Day 7)

- If using n8n self-hosted: deploy on a VPS (DigitalOcean, Railway, Render)
- Set all n8n workflows to `Active`
- Set Telegram webhook to production n8n URL
- Monitor n8n execution logs for first 48 hours

Message Templates

Welcome Message (step_0)

```
 Welcome to FlightTracker Bot!

I'll monitor flight prices for you and alert you when they drop.

 First — do you want to receive email alerts?
Reply: YES or NO
```

Confirmation Message (step_9)

```
✅ Got it! Here's your tracking setup:

✈  Toronto (YYZ) → Mumbai (BOM)
   Depart: 2026-08-15  |  Return: 2026-08-30
   1 Adult
   Alert when price drops by 10%
   Email alerts: YES (user@gmail.com)
   Nearby airports: YES

 Searching for current prices...
```

Current Prices Response

```
 Current Best Prices:

1. CAD 812 — Air Canada + Air India (AC872 + AI101)
2. CAD 845 — Emirates (EK241 + EK564)
3. CAD 890 — British Airways (BA092 + BA138)

 Tracking active! I'll alert you hourly if prices drop.
Send /stop to stop tracking | /status for latest prices
```

Price Drop Alert

```
 Price Drop Alert!

✈ Toronto (YYZ) → Mumbai (BOM)
 Aug 15, 2026

Was: CAD 812
Now: CAD 741 (↓ 8.7%)

Airline: Air India (AI Direct)
Book quickly — prices can change!
```

Best Deal Alert (3+ days)

```
⭐ BEST PRICE in 5 Days of Tracking!

✈ Toronto (YYZ) → Mumbai (BOM)
📅 Aug 15, 2026

Lowest Ever: CAD 698 (was CAD 820 when you started)
That's a saving of CAD 122! (↓ 14.9%)

⚡ This is the lowest we've seen — act fast!
```

Cost Summary

Service	Free Tier	Estimated Monthly Cost
Amadeus API (Test)	Unlimited	$0
Amadeus API (Production)	500 calls/month	$0–$10
Telegram Bot API	Unlimited	$0
Twilio WhatsApp	$0 incoming	~$1–$5 per 100 alerts
Gmail (via n8n)	Free	$0
Google Sheets API	Free	$0
n8n Cloud Starter	5,000 executions	$20/month
n8n Self-hosted (VPS)	—	~$5–$10/month

Total: ~$5–$30/month depending on volume and hosting

Claude Code Prompt to Use

When handing this to Claude Code, use this prompt:

> "I am building a flight price tracking automation using n8n. I have a detailed architecture document. My stack is:
>
> - n8n for automation workflows (self-hosted)
> - Telegram Bot API for the conversational interface
> - Amadeus Flight Offers Search API for flight prices
> - Google Sheets as the database (Sessions, Tracker, PriceHistory tabs)
> - Twilio WhatsApp API for WhatsApp alerts
> - Gmail OAuth via n8n for email alerts
>
> Please help me build:
>
> 1. The n8n workflow JSON for Workflow 1 (conversation bot with step-based state machine)
> 2. The n8n workflow JSON for Workflow 2 (hourly price monitor with drop detection)
> 3. The n8n workflow JSON for Workflow 3 (daily digest)
> 4. All JavaScript Code node logic for: state management, price comparison, nearby airport lookup, and message formatting
>
> Use the architecture document I've provided. Start with Workflow 1 first."